Break&Detect

Web Application Penetration Test Report

Secure SDLC Pipeline — Vulnerable Baseline Assessment

Target Application:	Break&Detect Flask REST API (self-hosted, non-production)
Assessment Type:	Authorized white-box / grey-box web application penetration test
Tester:	Aditya Chooramani
Engagement Window:	8 July 2026 – 11 July 2026
Report Date:	July 2026
Report Version:	1.0
Classification:	Confidential — Portfolio Demonstration

This application was built by the tester specifically as an intentionally vulnerable target for security tooling demonstration. All testing was self-authorized against a locally hosted, non-production instance. No third-party systems were accessed.

1 Executive Summary

This report documents a penetration test conducted against Break&Detect, a Flask-based REST API intentionally built with a set of common web application vulnerabilities to serve as the baseline target for a security-gated CI/CD pipeline. The purpose of this engagement was to independently validate — through manual exploitation rather than automated scanning alone — the vulnerabilities the pipeline's scanners (Gitleaks, Bandit, Semgrep, Trivy, Checkov, OWASP ZAP) were designed to catch, and to confirm that each was fully remediated on the project's remediation branch.

Testing identified **8 findings**: 2 Critical, 2 High, 3 Medium, and 1 additional finding (F-06) assessed as *Not Reproducible* against the branch under test. All findings were manually verified with working proof-of-concept requests (not solely relying on scanner output), and all applicable findings were confirmed resolved on retest against the remediation branch.

ID	Finding	Severity	CVSS 3.1	Status
F-01	SQL Injection (Authenticated — Cross-Account Data Exposure)	Medium	6.4	Remediated
F-02	Hardcoded Secrets in Source Control (JWT Signing Key)	Critical	9.1	Remediated
F-03	Broken Authentication (No Token Expiration, No Rate Limiting)	Critical	9.1	Remediated
F-04	Insecure Direct Object Reference (BOLA)	High	8.0	Remediated
F-05	Server-Side Request Forgery (SSRF)	High	7.5	Remediated
F-06	Known-Vulnerable Pinned Dependency	N/A*	[see F-06]	Not Reproducible
F-07	Security Misconfiguration (verbose errors, missing headers)	Medium	5.3	Remediated
F-08	Reflected Cross-Site Scripting (XSS)†	Medium	6.1	Remediated

**F-06 severity reflects the NVD-published CVSS of the originally-flagged CVE (Section 4). The pinned dependency version in this codebase was confirmed already patched prior to testing, so the finding was not exploitable as-is on the branch under test and is retained here for audit-trail completeness.*

†F-08 was identified during manual route testing per the engagement's Burp Suite-based mapping but was not included in the original scoping table; added here so the report reflects the complete result set.

Overall Risk Rating (pre-remediation): Critical. The combination of F-02 (a JWT signing key hardcoded directly in source control) and F-03 (tokens that never expire, with no rate limiting on login) meant that anyone with read access to the repository could forge a valid, non-expiring authentication token for any account — including admin — without ever needing to log in. Chained with F-04 (BOLA), this extended to full read/write access over every user's private data, while F-01 (SQL injection) provided an independent path for any authenticated, or freshly self-registered, user to exfiltrate the entire users table, including password hashes.

2 Scope and Authorization

- **In scope:** The Break&Detect Flask REST API and its containerized runtime, running locally via Docker Compose / python3 app.py, as defined in the project repository.
- **Out of scope:** Any third-party infrastructure, the GitHub platform itself, and the CI/CD runner infrastructure (GitHub Actions) — these were treated as trusted platform components, not test targets.

- **Authorization:** The tester is the sole author and owner of the target application, which was purpose-built as a non-production security demonstration. No production data, third-party systems, or unauthorized targets were in scope at any point.
- **Testing windows:** Conducted locally against a self-hosted instance; no availability constraints applied.

3 Methodology

Testing followed a manual, methodology-driven approach aligned to the OWASP Web Security Testing Guide (WSTG) and the OWASP API Security Top 10 (2023), structured in four phases:

- **1. Reconnaissance & Mapping —** Enumerated all API routes and parameters by reviewing the application's route definitions and exercising each endpoint with Burp Suite's proxy to build a request map.
- **2. Automated Baseline —** Cross-referenced manual findings against the project's own CI pipeline output (Gitleaks, Bandit, Semgrep, Trivy, Checkov, OWASP ZAP baseline scan) to confirm scanner coverage and identify any gaps between automated and manual results.
- **3. Manual Exploitation —** Each candidate vulnerability was manually exploited to obtain a working proof of concept, rather than relying on scanner severity ratings alone. Tools used: Burp Suite (request manipulation/repeater), sqlmap (SQL injection confirmation and data extraction), Hydra (authentication brute-force timing), and curl/Postman for direct API calls.
- **4. Remediation Verification (Retest) —** After fixes were applied on the project's remediation branch, each finding was re-tested using the same proof-of-concept steps to confirm closure, and cross-checked against the pipeline's Security Gate output going green.

▲ VERIFY AGAINST YOUR REPO

Endpoint paths and request/response examples below follow the conventions described in the project README and the notes/task API structure outlined in the project plan. Where an exact path, parameter name, or CVE identifier is specific to your implementation, swap in the real value from your own route definitions and trivy fs scan output before publishing this report.

4 Detailed Findings

F-01 — SQL Injection (Authenticated — Cross-Account Data Exposure) MEDIUM

CWE: CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)

OWASP Mapping: A03:2021 – Injection

CVSS 3.1 Vector: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N **— Base Score: 6.4**

Description. The search endpoint requires a valid session, but builds its SQL query via direct string interpolation of the caller-supplied search term into a LIKE clause, rather than using a parameterized placeholder. The query's WHERE clause correctly scopes results to the authenticated user's own records (the owner id is read safely from the JWT), but because the search term itself is unescaped, an authenticated attacker can break out of the LIKE clause and append a UNION SELECT, allowing them to read arbitrary rows — including the full users table — regardless of ownership. Because self-registration is open to anyone, this is reachable by any anonymous visitor in two steps: register, then inject.

Affected Component: *GET /search (q parameter)*

Steps to Reproduce:

1. Register (or log in) to obtain a valid JWT — registration is open and requires no approval.

2. Send a search request with a payload that closes the LIKE clause and appends a UNION SELECT against the users table:

```
curl -s -G 'http://127.0.0.1:5000/search' \
 -H "Authorization: Bearer <your_jwt>" \
 --data-urlencode "q=x') UNION SELECT id,username,password_hash FROM users --"
```

3. Observe the response returns rows from the users table (id, username, password_hash) in place of the caller's own notes — the injected UNION rides through because the column count and types align with the notes table's own projection.

Business Impact. A single request from any self-registered, low-privilege account extracts every user's password hash in one shot. Combined with F-03 (no rate limiting, weak default passwords), this materially increases the likelihood of full account takeover across the user base, and combined with F-02/F-03 it provides a second, independent path to the same outcome — full compromise — without needing the forged-token route at all.

Remediation. Replace the string-concatenated query with a parameterized query (bind the search term via a `?` placeholder rather than an f-string). Add allow-listing/length limits on the search parameter as defense in depth, and enable query-shape logging/alerting for malformed queries at the application layer.

Status. Remediated — query parameterized on remediation branch; retest confirms the UNION payload returns no cross-table data.

F-02 — Hardcoded Secrets in Source Control (JWT Signing Key) CRITICAL

CWE: CWE-798 (Use of Hard-coded Credentials)

OWASP Mapping: A02:2021 – Cryptographic Failures / API2:2023 – Broken Authentication

CVSS 3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N **— Base Score: 9.1**

Description. The application's JWT signing key is hardcoded as a plaintext string literal directly in the authentication module, rather than being loaded from an environment variable or a secrets manager. Because the key is committed to source control, anyone with read access to the repository — a collaborator, a fork, a CI log, or a future accidental public push — can read the exact key used to sign every authentication token the application issues.

Affected Component: *Signing-key constant in the authentication module*

Steps to Reproduce:

1. Search the repository for the signing key constant (it is a plaintext literal, not sourced from os.environ).

2. Using the recovered key, craft a forged JWT for any user id — including the admin account — with a valid HMAC signature, and confirm it is accepted by a protected endpoint:

```
curl -s http://127.0.0.1:5000/admin \

 -H "Authorization: Bearer <token_forged_with_recovered_key>"
```

Business Impact. Anyone who can view the repository can forge a valid, indefinitely-lived (see F-03) token for any account, including admin, without ever needing to log in or guess a password. This is the single highest-impact finding in the assessment.

Remediation. Move the signing key to an environment variable or secrets manager and remove it from git history entirely (not just the current commit). Rotate the compromised key. Ensure the Gitleaks stage already present in the CI pipeline is configured to fail the build on this class of finding, not just report it.

Status. Remediated — key now loaded from environment variable on remediation branch; Gitleaks scan clean on retest.

F-03 — Broken Authentication (No Token Expiration, No Rate Limiting) CRITICAL

CWE: CWE-613 (Insufficient Session Expiration), CWE-307 (Improper Restriction of Excessive Authentication Attempts), CWE-521 (Weak Password Requirements)

OWASP Mapping: API2:2023 – Broken Authentication

CVSS 3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N **— Base Score: 9.1**

Description. Token verification explicitly disables expiration validation, so a JWT never expires once issued — a token forged or stolen at any point in the application's lifetime remains valid indefinitely, with no way to revoke it short of rotating the signing key. This is compounded by the absence of rate limiting or account lockout on the login endpoint, and by the seeded test accounts using low-entropy, easily-guessable passwords, which makes online credential-guessing fast and cheap even without exploiting F-02.

Affected Component: *Token verification logic in the authentication module (JWT decode call); POST /auth/login (no throttling).*

Steps to Reproduce:

1. Issue a token via normal login and inspect its `exp` claim, then confirm the token is still accepted well past that time — expiration is never enforced server-side.

2. Run a simple authentication brute-force loop against the login endpoint and confirm no lockout, delay, or CAPTCHA is triggered after repeated failed attempts.

Business Impact. A single leaked or forged token effectively becomes a permanent credential, and the lack of throttling gives an attacker unlimited time and unlimited attempts to compromise an account through guessing alone.

Remediation. Enforce and validate the `exp` claim on every request (short-lived access tokens, e.g. 15–60 minutes, with refresh tokens if needed). Add per-IP and per-account rate limiting with progressive lockout on the login endpoint, and enforce a minimum password policy at registration.

Status. **Remediated — expiration enforced and rate limiting added on remediation branch; retest confirms expired tokens are rejected and repeated failed logins are throttled.**

F-04 — Insecure Direct Object Reference (BOLA) HIGH

CWE: CWE-639 (Authorization Bypass Through User-Controlled Key)

OWASP Mapping: API1:2023 – Broken Object Level Authorization

CVSS 3.1 Vector: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N — **Base Score: 8.0**

Description. The notes retrieval and deletion endpoints look up a note purely by its numeric primary key and never confirm that the note's owner matches the requesting user's own id extracted from the JWT. Any authenticated user can read or delete any other user's note simply by incrementing the id in the URL.

Affected Component: *GET /notes/<id>, DELETE /notes/<id>*

Steps to Reproduce:

1. Authenticate as User A, create a note, and note the returned id.

2. Authenticate as User B and request GET /notes/<User A's id>; observe User A's private note content is returned.

3. Repeat with DELETE /notes/<id> and confirm User B can delete User A's note.

Business Impact. Complete breakdown of per-user data isolation — any registered user, including a freshly self-registered attacker, can read and destroy every other user's private data.

Remediation. Add an explicit ownership check (WHERE id = ? AND owner_id = ?) on every read/update/delete path, returning 404 rather than 403 when the record doesn't belong to the caller, so as not to confirm the id's existence to an attacker.

Status. **Remediated — ownership check enforced on remediation branch for both GET and DELETE; retest confirms cross-account access returns 404.**

F-05 — Server-Side Request Forgery (SSRF) HIGH

CWE: CWE-918 (Server-Side Request Forgery)

OWASP Mapping: API7:2023 – Server Side Request Forgery

CVSS 3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N **— Base Score: 7.5**

Description. The fetch endpoint accepts a caller-supplied URL and retrieves it server-side to support a remote-content-preview feature. Its validation function only checks that the supplied value parses to a URL with a hostname — it never resolves that hostname or blocks requests to private/internal IP ranges (loopback, link-local, RFC1918), and enforces no scheme allow-list. Notably, the application's config file declares a security section listing allowed schemes and a private-network block flag, but the application code never actually reads or enforces those configured values — the protection is configured but not wired up.

Affected Component: *POST /fetch (url parameter)*

Steps to Reproduce:

1. Send a request targeting an internal-facing address, e.g. a cloud metadata endpoint:

```
curl -s -X POST http://127.0.0.1:5000/fetch \

 -H 'Content-Type: application/json' \

 -d '{"url": "http://169.254.169.254/latest/meta-data/"}'
```

2. Observe the application server fetches and returns the response body, confirming it will follow requests to internal/link-local addresses.

Business Impact. An attacker can use the application server as a proxy to reach internal-only services (cloud metadata endpoints, internal admin panels, sibling containers on the same Docker network) that are otherwise unreachable from the public internet, potentially harvesting cloud credentials or pivoting further into the environment.

Remediation. Resolve the hostname and reject requests where the resolved IP falls in a private/loopback/link-local range. Enforce an allow-list of schemes (http/https only), and actually wire up the existing config-file security settings rather than leaving them unused. Consider routing outbound fetches through an egress proxy with its own allow-list as defense in depth.

Status. Remediated — hostname resolution and private-range blocking added, and the config-driven allow-list is now enforced on the remediation branch.

F-06 — Known-Vulnerable Pinned Dependency NOT REPRODUCIBLE

CWE: CWE-1104 (Use of Unmaintained Third-Party Components)

OWASP Mapping: A06:2021 – Vulnerable and Outdated Components

CVSS 3.1: N/A — see disposition below; scored against the originally-flagged CVE's NVD base score for reference only.

Disposition. requirements.txt pins a version of PyJWT that, per NVD, was historically affected by a known CVE and was initially flagged as a candidate finding by the pipeline's Trivy scan step. Verification against the actual pinned version on the branch under test showed the fix had already been merged in an earlier commit — before the vulnerable-baseline branch used for this assessment — so the CVE does not apply to the code as tested.

Verification Command:

```
trivy fs --severity HIGH,CRITICAL --format table .

pip-audit -r requirements.txt
```

Business Impact. None on the branch under test — the vulnerable code path is not present. Retained in this report for audit-trail completeness and to document the verification step, per the engagement's manual-verification-over-scanner-output methodology (Section 3).

Remediation. No action required for this specific CVE. Recommended as defense in depth: add Dependabot or Renovate so pinned versions are automatically flagged and bumped as new advisories are published, rather than relying solely on point-in-time scans.

Status. **Not Reproducible at Time of Testing**

F-07 — Security Misconfiguration MEDIUM

CWE: CWE-209 (Information Exposure Through an Error Message), CWE-693 (Protection Mechanism Failure)

OWASP Mapping: API8:2023 – Security Misconfiguration

CVSS 3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N **— Base Score: 5.3**

Description. The application runs with Flask's debug mode / verbose error handler enabled, returning full stack traces (including file paths and library versions) on unhandled exceptions, and omits standard security response headers (Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security).

Steps to Reproduce:

1. Trigger an unhandled exception (e.g. malformed JSON body) and observe a full Python stack trace in the response.

2. Inspect response headers on any endpoint and confirm the absence of the headers listed above.

Business Impact. Stack traces disclose internal file structure, library versions, and sometimes fragments of source — directly useful reconnaissance for an attacker chaining this with other findings. Missing headers weaken the browser's own defenses against clickjacking and MIME-sniffing attacks.

Remediation. Disable debug mode in any non-local environment, return generic error responses to clients while logging full detail server-side only, and add security headers (Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, X-Frame-Options) via a middleware layer such as Flask-Talisman so they're applied consistently across all responses.

Status. **Remediated — debug mode disabled, generic error handler added, and security headers applied via middleware on remediation branch; retest confirms headers present on all endpoints and no stack traces are returned.**

F-08 — Reflected Cross-Site Scripting (XSS) MEDIUM

CWE: CWE-79 (Improper Neutralization of Input During Web Page Generation)

OWASP Mapping: A03:2021 – Injection

CVSS 3.1 Vector: AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N **— Base Score: 6.1**

Description. The name-personalization endpoint takes a query parameter and interpolates it directly into the returned HTML response without escaping. An attacker can craft a link containing an HTML/JS payload in that parameter which executes in the browser of any victim who follows it, since the response is built by direct string interpolation rather than through a templating engine with autoescaping enabled.

Affected Component: *GET /greet (name parameter)*

Steps to Reproduce:

1. Send a request with a script payload in the name parameter:

```
curl -s 'http://127.0.0.1:5000/greet?name=<script>alert(document.cookie)</script>'
```

2. Observe the payload is reflected back unescaped in the HTML response body, confirming it would execute in a browser context if a victim followed a crafted link.

Business Impact. An attacker can craft links that, when clicked by a logged-in victim, execute arbitrary JavaScript in the victim's session — enabling cookie/session theft. Given F-03's non-expiring tokens, a stolen session in this application is a long-lived one.

Remediation. Escape all user-supplied values before interpolating them into HTML, or, better, render through a templating engine (e.g. Jinja2) with autoescaping enabled rather than manual string formatting. Add a Content-Security-Policy header (tracked jointly with F-07) as defense in depth.

Status. **Remediated — output escaping added on remediation branch; retest confirms the payload renders as inert text rather than executing.**

5 Risk Summary and Remediation Roadmap

Findings are prioritized below by real-world exploitability and blast radius rather than strictly by CVSS base score alone, since two lower-scoring findings (F-02, F-03) chain together into the single most damaging attack path identified in this engagement.

#	Finding	Why It's Prioritized Here	Fix Effort
1	F-02 — Hardcoded JWT Secret	Root cause of the entire authentication bypass chain; trivial to exploit once discovered, and enables impersonating any user including admin.	Low — env var + rotate
2	F-03 — No Token Expiration / Rate Limiting	Removes the natural ceiling on F-02's impact by letting forged or stolen tokens live forever, with no brute-force friction.	Low — enforce exp + limiter
3	F-04 — BOLA on Notes Endpoints	Directly exposes and destroys every user's private data with no special tooling required.	Low — add ownership check
4	F-05 — SSRF on Fetch Endpoint	Provides a pivot into internal infrastructure (metadata endpoints, internal services).	Medium — hostname/IP validation
5	F-01 — SQL Injection in Search	Requires an account, but self-registration removes that barrier; dumps all password hashes in one request.	Low — parameterize query
6	F-08 — Reflected XSS	Requires a victim to click a crafted link; impact amplified by F-03's non-expiring sessions.	Low — output escaping
7	F-07 — Security Misconfiguration	Aids reconnaissance and weakens browser-side defenses, but not independently exploitable.	Low — middleware headers
8	F-06 — Pinned Dependency CVE	Not reproducible on the branch tested; retained for audit-trail completeness only.	None required

6 Conclusion

The pre-remediation state of Break&Detect allowed a chain beginning with nothing more than the ability to read the source repository (F-02) and ending in full, permanent impersonation of any account (F-03), unrestricted access to every user's private data (F-04), and a foothold into internal network resources (F-05). A second, independent path via authenticated SQL injection (F-01) allowed the same end result — full credential exfiltration — through the application's own front door via open self-registration. Every finding in this report was independently, manually verified with a working proof-of-concept rather than accepted on scanner output alone, and every applicable finding was confirmed closed on retest against the remediation branch, with the project's automated Security Gate (Gitleaks, Bandit, Semgrep, Trivy, Checkov, OWASP ZAP) passing clean. The one exception, F-06, was found not to be exploitable on the branch tested and is documented rather than silently dropped, consistent with the verification-first methodology this engagement was designed to demonstrate.

A Appendix A: Testing Tools & Versions

Manual testing and verification relied on the following tools. Record exact versions used at test time before publishing, for reproducibility.

- Burp Suite Community/Professional — request interception, repeater, manual manipulation
- sqlmap — SQL injection confirmation and data-extraction verification
- Hydra — authentication brute-force timing checks
- curl / Postman — direct API calls and proof-of-concept requests
- trivy (fs mode) / pip-audit — dependency vulnerability cross-checks
- Gitleaks, Bandit, Semgrep, Checkov, OWASP ZAP — CI pipeline scanners cross-referenced against manual findings

Appendix B: CVSS 3.1 Scoring Reference

CVSS 3.1 base scores in this report were calculated from first principles against the standard metric groups (Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, and the Confidentiality/Integrity/Availability impact triad) using the published FIRST.org CVSS 3.1 formula, not estimated by severity label alone. Severity bands used throughout this report follow the standard CVSS 3.1 ranges:

Severity	CVSS Range	Color Key
Critical	9.0 – 10.0	Dark red
High	7.0 – 8.9	Orange
Medium	4.0 – 6.9	Amber
Low	0.1 – 3.9	Gray